

Studiu experimental comparativ asupra algoritmilor de sortare

David Carlos Necşoiu
Universitatea de Vest din Timișoara
david.necsoiu06@e-uvt.ro

2026

Rezumat

Lucrarea prezintă o analiză teoretică și experimentală a cinci algoritmi de sortare: Bubble Sort, Insertion Sort, Selection Sort, Merge Sort și Counting Sort. Problema studiată este alegerea unui algoritm potrivit în funcție de dimensiunea datelor, distribuția valorilor și costul de execuție. Studiul pornește de la literatura de specialitate, evaluează soluții existente și propune un cadru experimental propriu, implementat în limbajul C. Cadrul generează mai multe distribuții de date, măsoară timpul mediu de execuție, verifică sortarea și exportă rezultatele în format CSV. Ca extensie opțională, este inclusă o variantă paralelă pentru Merge Sort folosind OpenMP. Rezultatele confirmă diferența dintre algoritmii cu complexitate pătratică, algoritmii de tip divide et impera și algoritmii care nu se bazează exclusiv pe comparații.

Cuprins

1	Introducere	3
1.1	Problema studiată	3
1.2	Motivația lucrării	3
2	Analiza audienței și obiective operaționale	3
2.1	Analiza audienței	3
2.2	Obiective operaționale	4
3	Căutare în literatura de specialitate	4
4	Evaluarea soluțiilor existente	5
4.1	Algoritmi simpli	5
4.2	Algoritmi eficienți prin comparații	5
4.3	Algoritmi care nu se bazează pe comparații	5
4.4	Soluții de bibliotecă	5
5	Soluția proprie: modelare și implementare	6
5.1	Modelarea problemei	6
5.2	Tipuri de date generate	6
5.3	Măsurarea timpului	6

5.4	Verificarea corectitudinii	6
5.5	Extensia paralelă: OpenMP	7
6	Experimentul	7
6.1	Metodologia experimentală	7
6.2	Mediul de testare	7
6.3	Rezultate obținute în proiectul inițial	8
6.4	Interpretarea rezultatelor	8
7	Comparația cu literatura de specialitate	8
8	Limitări	9
9	Concluzii și direcții viitoare	9

1 Introducere

Sortarea este o problemă fundamentală în informatică. Un algoritm de sortare primește o secvență de elemente și produce o permutare a acestora într-o anumită ordine, de obicei crescătoare. Importanța sortării apare deoarece multe alte operații devin mai eficiente atunci când datele sunt ordonate: căutarea, interclasarea, eliminarea duplicatelor sau generarea de rapoarte.

1.1 Problema studiată

Problema lucrării poate fi formulată astfel: pentru un vector de numere întregi de dimensiune n , se dorește determinarea comportamentului mai multor algoritmi de sortare în funcție de distribuția datelor de intrare. Intrarea este un vector $v = (v_1, v_2, \dots, v_n)$, iar ieșirea este o permutare a aceluiași vector astfel încât $v_1 \leq v_2 \leq \dots \leq v_n$.

Lucrarea nu urmărește doar implementarea unor algoritmi, ci și înțelegerea condițiilor în care un algoritm este preferabil altuia. De exemplu, Insertion Sort poate fi eficient pentru date aproape sortate, dar devine lent pe date sortate descrescător. Merge Sort are un comportament mai stabil, dar folosește memorie suplimentară. Counting Sort este foarte rapid când intervalul valorilor este mic, dar poate deveni nepractic dacă intervalul este foarte mare [1, 2].

1.2 Motivația lucrării

Motivația principală este legată de faptul că sortarea este una dintre primele probleme prin care se poate observa diferența dintre un algoritm ușor de scris și un algoritm eficient. Pentru un număr mic de elemente, diferențele pot părea nesemnificative. Pentru un număr mare de elemente, alegerea algoritmului poate schimba timpul de execuție de la milisecunde la secunde sau chiar minute.

Această lucrare are și un scop didactic: cititorul trebuie să poată urmări legătura dintre teoria complexității și rezultatele experimentale obținute prin rularea codului.

2 Analiza audienței și obiective operaționale

2.1 Analiza audienței

Lucrarea este adresată unei audiențe formate din studenți și cadre didactice din domeniul informaticii. Din acest motiv, explicațiile sunt păstrate la un nivel accesibil, dar folosesc noțiuni formale precum complexitate asimptotică, caz favorabil, caz mediu și caz defavorabil.

Tabela 1: Analiza audienței

Criteriu	Interpretare pentru această lucrare
Nivel de cunoștințe	Cititorul cunoaște noțiuni de bază despre vectori, funcții, recursivitate și complexitate.
Interes	Interesul principal este înțelegerea diferenței dintre algoritmi și justificarea alegerii unuia într-un context concret.
Așteptări	Cititorul se așteaptă la o lucrare clar structurată, cu surse citate, metodologie reproductibilă, rezultate și concluzii.

2.2 Obiective operaționale

Obiectivele sunt formulate astfel încât la finalul lucrării cititorul să poată demonstra că a înțeles atât partea teoretică, cât și partea practică:

1. să definească formal problema sortării;
2. să explice diferența dintre algoritmi de complexitate $O(n^2)$, $O(n \log n)$ și $O(n + k)$;
3. să identifice situațiile în care un algoritm simplu poate fi suficient;
4. să compare rezultatele experimentale cu rezultatele teoretice din literatura de specialitate;
5. să descrie contribuția proprie: cadrul de testare, verificarea corectitudinii, exportul CSV și opțiunea de paralelizare.

Din punct de vedere pedagogic și cognitiv, lucrarea urmărește trei etape: **motivație**, prin explicarea importanței sortării; **învățare**, prin prezentarea algoritmilor și a metodologiei; **asociere**, prin legarea rezultatelor experimentale de teoria din literatura de specialitate.

3 Căutare în literatura de specialitate

Literatura de specialitate tratează sortarea ca o temă centrală în proiectarea și analiza algoritmilor. Cormen et al. prezintă algoritmi precum Insertion Sort, Merge Sort, Heap Sort, Quick Sort și Counting Sort, punând accent pe corectitudine și complexitate [1]. Knuth discută sortarea într-un cadru mai larg, cu accent pe metode clasice și analiză detaliată [2]. Wirth pune accent pe legătura dintre algoritmi și structurile de date folosite în implementare [3].

Tabela 2: Complexități teoretice ale algoritmilor studiați

Algoritm	Favorabil	Mediu	Defavorabil	Memorie	Referință
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	[2, 3]
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	[1, 3]
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	[2, 3]
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	[1]
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	[1]

O observație importantă este că nu toți algoritmii pot fi comparați doar prin notația Big-O. Distribuția datelor, constantele ascunse, accesul la memorie și dimensiunea intervalului valorilor influențează rezultatele practice. De aceea, partea experimentală este necesară pentru a verifica dacă rezultatele obținute în implementare sunt compatibile cu teoria.

4 Evaluarea soluțiilor existente

4.1 Algoritmi simpli

Bubble Sort, Insertion Sort și Selection Sort sunt algoritmi simpli, potriviți pentru învățare. Avantajul lor este claritatea implementării. Dezavantajul este complexitatea pătratică, care devine problematică pentru vectori mari [2, 3]. Acești algoritmi sunt incluși în lucrare deoarece permit observarea directă a diferenței dintre cod simplu și performanță.

4.2 Algoritmi eficienți prin comparații

Merge Sort și Quick Sort sunt exemple de algoritmi eficienți bazați pe comparații. Quick Sort are performanță medie bună, dar poate avea caz defavorabil $O(n^2)$. Merge Sort are avantajul unui timp $O(n \log n)$ în toate cazurile principale, dar are nevoie de memorie suplimentară [1]. În această lucrare a fost ales Merge Sort deoarece comportamentul său este stabil și este ușor de extins la o variantă paralelă.

4.3 Algoritmi care nu se bazează pe comparații

Counting Sort este eficient atunci când valorile sunt întregi și aparțin unui interval limitat. Acesta poate depăși limita inferioară $\Omega(n \log n)$ valabilă pentru sortările generale prin comparații, deoarece folosește informația suplimentară despre intervalul valorilor [1]. Limitarea sa este consumul de memorie $O(k)$, unde k este dimensiunea intervalului valorilor.

4.4 Soluții de bibliotecă

În limbajul C există și soluții de bibliotecă, precum funcția `qsort`. Totuși, scopul acestor lucrări nu este doar obținerea celui mai mic timp, ci înțelegerea mecanismului fiecărui algoritm. Din acest motiv, algoritmii au fost implementați explicit, iar funcțiile de bibliotecă pot fi considerate o direcție viitoare de comparație.

5 Soluția proprie: modelare și implementare

Contribuția proprie a lucrării nu constă în inventarea algoritmilor, ci în construirea unui cadru experimental coerent și reproductibil. Acest cadru permite testarea tuturor algoritmilor pe aceleași tipuri de date, folosind aceeași metodă de măsurare.

5.1 Modelarea problemei

Fiecare algoritm primește aceeași interfață:

Listing 1: Interfață comună pentru algoritmii de sortare

```
1 typedef void (*SortFunction)(int *v, int n);
```

Această interfață permite apelarea algoritmilor într-un mod uniform. Programul poate păstra o listă de algoritmi, fiecare având un nume și o funcție asociată. Astfel, adăugarea unui algoritm nou presupune doar implementarea funcției și introducerea ei în listă.

5.2 Tipuri de date generate

Pentru a evita concluziile bazate pe un singur caz, au fost folosite cinci tipuri de date:

1. **date aleatoare**, pentru un caz general;
2. **date sortate crescător**, pentru a observa cazurile favorabile;
3. **date sortate descrescător**, pentru a observa cazurile defavorabile;
4. **date aproape sortate**, pentru a testa algoritmi sensibili la ordinea inițială;
5. **liste plate**, cu multe valori egale, pentru a observa comportamentul pe date cu redundanță mare.

5.3 Măsurarea timpului

Pentru fiecare test, vectorul inițial este copiat într-un vector de lucru. Astfel, fiecare algoritm primește exact aceleași valori, iar rezultatele sunt comparabile. Timpul raportat este media mai multor rulări. Se salvează și timpul minim și maxim, deoarece acestea pot arăta variații produse de sistemul de operare sau de alte procese.

Listing 2: Ideea de măsurare: copiere, sortare, verificare

```
1 copy_array(work, base, n);  
2 start = now_seconds();  
3 function(work, n);  
4 finish = now_seconds();  
5 sorted_ok = is_sorted_ascending(work, n);
```

5.4 Verificarea corectitudinii

Un timp mic nu este relevant dacă rezultatul nu este corect. Din acest motiv, după fiecare rulare se verifică dacă vectorul este sortat crescător. Rezultatul verificării este salvat în fișierul CSV prin coloana **sorted_ok**.

5.5 Extensia paralelă: OpenMP

Ca extensie opțională, proiectul include o variantă de Merge Sort paralel. OpenMP este un model de programare paralelă pentru sisteme cu memorie partajată, folosit în C, C++ și Fortran [4]. În implementare, paralelizarea este activată doar dacă programul este compilat cu opțiunea `-fopenmp`. Dacă compilatorul nu suportă OpenMP, proiectul rămâne funcțional fără partea paralelă.

Listing 3: Compilare fără OpenMP

```
1 gcc SA_improved.c -O2 -std=c11 -o sortare.exe
```

Listing 4: Compilare cu OpenMP, dacă este disponibil

```
1 gcc SA_improved.c -O2 -std=c11 -fopenmp -o sortare.exe
```

Paralelizarea nu este aplicată orbește pentru orice subproblemă. Pentru dimensiuni mici, costul creării task-urilor poate fi mai mare decât beneficiul. De aceea, implementarea folosește un prag minim și o adâncime maximă pentru task-uri.

6 Experimentul

6.1 Metodologia experimentală

Experimentul are următorii pași:

1. se alege dimensiunea vectorului n ;
2. se generează un set de date pentru fiecare distribuție;
3. fiecare algoritm primește o copie a aceluiași set de date;
4. se execută mai multe rulări pentru fiecare combinație algoritm-distribuție;
5. se calculează timpul mediu, minim și maxim;
6. se verifică rezultatul sortării;
7. rezultatele sunt salvate în fișierul `results.csv`.

6.2 Mediul de testare

n

Pentru ca experimentul să fie reproductibil, trebuie precizat mediul de testare. Aceste date trebuie completate înainte de predare:

- Procesor: **CPU 11th Gen Intel(R) Core(TM) i7-11370H @ 3.30GHz**;
- Memorie RAM: **32.0 GB**;
- Sistem de operare: **Windows**;
- Compilator: **(MinGW.org GCC-6.3.0-1) 6.3.0**;

6.3 Rezultate obținute în proiectul inițial

Tabelul următor prezintă rezultatele existente în proiectul inițial, exprimate în miliseconde. După rularea versiunii îmbunătățite, tabelul poate fi înlocuit cu valorile generate în `results.csv`.

Tabela 3: Timpi existenți în proiectul inițial, în miliseconde

Algoritm	Aleator	Crescător	Descrescător	Aproape sortat	Liste plate
Bubble Sort	502.955	536.500	542.300	537.280	531.644
Merge Sort	2.673	1.467	1.492	1.536	1.381
Insertion Sort	150.691	0.063	302.114	0.119	0.058
Selection Sort	211.312	220.192	221.804	220.087	219.004
Counting Sort	0.517	0.657	0.669	0.646	0.603

6.4 Interpretarea rezultatelor

Rezultatele sunt compatibile cu teoria. Bubble Sort și Selection Sort au timpi mari deoarece execută multe comparații. Selection Sort face aproximativ același număr de comparații indiferent de ordinea inițială, motiv pentru care diferențele dintre distribuții nu sunt foarte mari. Bubble Sort poate fi optimizat cu oprire anticipată, dar rămâne ineficient pentru date mari.

Insertion Sort are comportament foarte bun pe date sortate sau aproape sortate, deoarece elementele sunt deja aproape de poziția lor finală. Pe date descrescătoare, fiecare element trebuie mutat pe o distanță mare, ceea ce explică timpul mai mare.

Merge Sort are timpi mici și relativ stabili. Acest comportament corespunde complexității $O(n \log n)$, care nu depinde puternic de ordinea inițială a datelor. Counting Sort este cel mai rapid în tabel deoarece valorile aparțin unui interval mic. Totuși, această concluzie trebuie formulată cu atenție: dacă intervalul valorilor devine foarte mare, memoria necesară pentru vectorul de frecvență poate deveni o problemă.

7 Comparația cu literatura de specialitate

Comparația cu literatura arată că rezultatele experimentale urmează tendințele teoretice. Conform literaturii, Bubble Sort, Insertion Sort și Selection Sort au complexitate pătratică în cazurile principale [2, 3]. În experiment, acești algoritmi au timpi semnificativ mai mari decât Merge Sort și Counting Sort.

Pentru Insertion Sort, literatura menționează eficiența pe date aproape sortate [1, 3]. Acest lucru apare și în tabel, unde Insertion Sort este mult mai rapid pe date sortate crescător și aproape sortate decât pe date aleatoare sau descrescătoare.

Pentru Merge Sort, teoria indică $O(n \log n)$ în caz favorabil, mediu și defavorabil [1]. Rezultatele experimentale arată o variație redusă între distribuții, ceea ce susține această proprietate.

Pentru Counting Sort, literatura arată că performanța depinde de n și de k , unde k este dimensiunea intervalului de valori [1]. Rezultatele bune obținute în experiment sunt explicate de faptul că intervalul valorilor este mic.

8 Limitări

Lucrarea are următoarele limitări:

- rezultatele depind de procesor, memorie, sistem de operare, compilator și procesele care rulează în fundal;
- numărul de rulări influențează stabilitatea mediei;
- Counting Sort este avantajos doar pentru intervale rezonabile de valori;
- OpenMP nu este disponibil în toate instalările de MinGW/GCC, deci partea paralelă trebuie tratată ca extensie opțională;
- nu au fost incluse toate sortările posibile, precum Heap Sort, Quick Sort sau sortarea standard din biblioteca C.

9 Concluzii și direcții viitoare

Lucrarea arată că alegerea algoritmului de sortare depinde de context. Pentru date mici sau pentru scopuri didactice, algoritmi simpli pot fi suficienți. Pentru date mari, algoritmi cu complexitate $O(n \log n)$, precum Merge Sort, sunt mai potriviți. Pentru valori întregi într-un interval mic, Counting Sort poate fi foarte eficient.

Contribuția proprie este cadrul experimental: generarea mai multor distribuții, măsurarea timpului mediu, verificarea corectitudinii, exportul CSV și includerea unei extensii paralele opționale. Această abordare transformă proiectul dintr-o simplă colecție de implementări într-un studiu comparativ.

Ca direcții viitoare se pot adăuga Quick Sort, Heap Sort, comparația cu `qsort`, grafice generate automat din CSV și teste pentru dimensiuni mai mari ale vectorilor. De asemenea, varianta OpenMP poate fi analizată separat pe sisteme cu mai multe nuclee, pentru a observa când paralelizarea produce un câștig real.

Referințe

- [1] Thomas H. Cormen et al., *Introduction to Algorithms*, a 3-a ed., MIT Press, 2009.
- [2] Donald E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, a 2-a ed., Addison-Wesley, 1998.
- [3] Niklaus Wirth, *Algorithms + Data Structures = Programs*, Prentice-Hall, 1976.
- [4] OpenMP Architecture Review Board, *OpenMP API Specification for Parallel Programming*, <https://www.openmp.org/>, Accesat la 6 mai 2026, 2026.